

TALLER PRÁCTICO

Terraform + Docker

Automatizando una arquitectura Node.js + React + Nginx + MySQL, localmente con Docker

Maestría en Ingeniería de Software · Desarrollo de Aplicaciones Empresariales

Guía para el estudiante

Contenido

Este índice se actualiza automáticamente al abrir el documento en Word. Si no aparece de inmediato, haz clic derecho sobre él y selecciona "Actualizar campo" (o presiona F9).

0. Objetivo del taller	4
1. Arquitectura que vas a construir	4
2. Prerrequisitos.....	5
3. Estructura del proyecto	5
4. Parte A — Entender las piezas de aplicación	6
4.1 Backend Node.js (backend/src/index.js).....	6
4.2 Frontend React (frontend/src/App.jsx).....	6
4.3 Nginx (nginx/nginx.conf.tpl)	6
4.4 MySQL (mysql/init.sql)	7
5. Parte B — El código Terraform, explicado	7
5.1 versions.tf	7
5.2 variables.tf	7
5.3 locals.tf.....	7
5.4 network.tf	8
5.5 mysql.tf.....	8
5.6 backend.tf	8
5.7 frontend.tf	8
5.8 nginx.tf.....	8
5.9 outputs.tf	9
6. Parte C — Ejecutar el taller paso a paso	9
Paso 1 — Inicializar	9
Paso 2 — Formatear y validar	9
Paso 3 — Planificar	9
Paso 4 — Aplicar	10
Paso 5 — Verificar.....	10
7. Parte D — El corazón del taller: practicar el archivo de estado.....	10
7.1 Inspeccionar el estado	10
7.2 Idempotencia: re-aplicar sin cambios	11
7.3 Provocar y detectar drift.....	11
7.4 Corregir el drift.....	12

7.5 Reemplazar un recurso puntual con -replace	12
7.6 Renombrar una dirección sin destruir infraestructura real	12
7.7 Importar un recurso creado manualmente	13
7.8 Resumen de la comparación.....	14
8. Parte E — Escalar el backend.....	15
9. Parte F — Destruir el entorno.....	15
10. Consejos si esto se llevara a producción.....	15
11. Solución de problemas comunes	17
12. Retos adicionales (para quien quiera ir más allá)	17
13. Referencia rápida de comandos	18

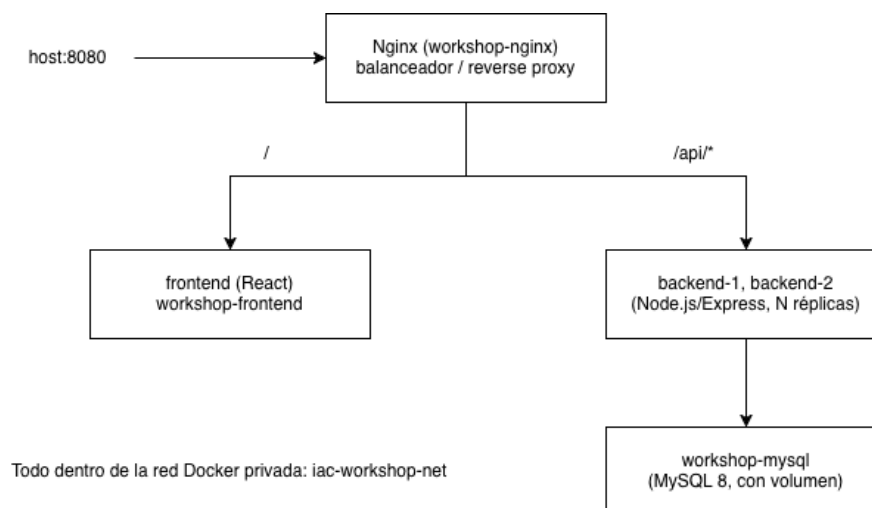
0. Objetivo del taller

Al finalizar, vas a poder:

1. Desplegar con Terraform una arquitectura de cuatro capas — **Nginx (balanceador) → React (frontend) → Node.js (backend, con réplicas) → MySQL (base de datos)** — usando el proveedor `docker` de Terraform en lugar de un proveedor de nube pública.
2. Ejecutar el flujo completo `init → plan → apply → destroy`.
3. **Practicar de forma guiada el archivo de estado (`terraform.tfstate`)**: inspeccionarlo, provocar y detectar drift, reemplazar recursos puntuales, renombrar direcciones sin destruir infraestructura, e importar un recurso creado manualmente — todo aquello que Docker Compose **no** puede hacer, porque no mantiene un state file equivalente.
4. Escalar el backend cambiando una sola variable y observar cómo Terraform recalcula el plan y regenera la configuración de Nginx.
5. Aplicar una lista concreta de recomendaciones antes de llevar este mismo patrón a producción real.

No se usa AWS, GCP ni Azure en ningún momento: todo corre en contenedores Docker locales, así que el taller no consume crédito de nube.

1. Arquitectura que vas a construir



- **Nginx** es el único servicio con un puerto publicado al host (`8080` por defecto). Reparte `/api/` entre las réplicas del backend (balanceo de carga round-robin) y sirve `/` proxy-eando al contenedor del frontend.
- **React** se compila en una imagen multi-stage y se sirve como archivos estáticos a través de su propio Nginx interno.

- **Node.js/Express** expone `/api/health` (para ver qué réplica respondió) y `/api/visits` (que lee y escribe en MySQL). El número de réplicas es una variable de Terraform.
- **MySQL** guarda sus datos en un volumen Docker nombrado, para que sobrevivan a recreaciones del contenedor.

2. Prerrequisitos

Herramienta	Versión mínima	Verificar con
Docker Desktop o Docker Engine	24.x	<code>docker version</code>
Terraform CLI	1.5.0	<code>terraform -version</code>
Node.js (opcional, solo si quieres correr frontend/backend fuera de Docker)	20.x	<code>node -v</code>

No necesitas cuenta en ningún proveedor cloud, ni tarjeta de crédito, ni variables de credenciales. El proveedor `docker` de Terraform habla directamente con el socket local de Docker (`/var/run/docker.sock` en Linux/Mac, `npipe` en Windows), el mismo socket que usa el propio comando `docker`.

Nota: aunque no se use la nube, `terraform init` sí necesita acceso a internet la primera vez, para descargar el plugin del proveedor `kreuzwerker/docker` desde el Registry de Terraform, y Docker necesita descargar las imágenes base (`node`, `nginx`, `mysql`) desde Docker Hub. Después de la primera ejecución, todo queda en caché local.

3. Estructura del proyecto

Este taller viene acompañado de un proyecto ya escrito y funcional (revisa el .zip adjunto). La estructura es:

```
workshop/
├── backend/
│   ├── src/index.js      ← API Express (health + visits)
│   ├── package.json
│   └── Dockerfile
├── frontend/
│   ├── src/App.jsx       ← UI React que consume la API
│   ├── src/main.jsx
│   ├── index.html
│   ├── package.json
│   ├── vite.config.js
│   └── Dockerfile        ← build multi-stage (Node → Nginx)
└── nginx/
    └── nginx.conf.tpl    ← plantilla que Terraform renderiza
```

```

├─ mysql/
│   └─ init.sql          ← esquema inicial de la base de datos
├─ terraform/
│   ├── versions.tf      ← provider + versión de Terraform
│   ├── variables.tf
│   ├── locals.tf
│   ├── network.tf
│   ├── mysql.tf
│   ├── backend.tf       ← usa "count" para las réplicas
│   ├── frontend.tf
│   ├── nginx.tf         ← usa templatefile() + upload
│   ├── outputs.tf
│   └─ terraform.tfvars.example

```

Solo el directorio `terraform/` contiene código Terraform; las carpetas `backend/`, `frontend/`, `nginx/` y `mysql/` son las aplicaciones y archivos de configuración que Terraform va a **construir y desplegar**, no a definir.

4. Parte A — Entender las piezas de aplicación

No es necesario escribir estos archivos desde cero — ya vienen en el proyecto — pero conviene leerlos antes de tocar Terraform, porque el código de infraestructura los referencia directamente.

4.1 Backend Node.js (`backend/src/index.js`)

Puntos clave para la clase:

- Usa `os.hostname()` para reportar qué contenedor atendió la petición. Terraform va a fijar ese hostname explícitamente (`backend-1`, `backend-2`...), así que al hacer varias peticiones a través de Nginx vas a ver el nombre cambiar — evidencia visual directa del balanceo de carga.
- La conexión a MySQL se arma **100% desde variables de entorno** (`DB_HOST`, `DB_USER`, etc.). El backend no sabe ni le importa si esas variables vienen de Terraform, de Docker Compose o de un `.env` — esta separación es la que permite que la misma imagen sirva para dev y para producción, cambiando solo la configuración externa.

4.2 Frontend React (`frontend/src/App.jsx`)

- Hace `fetch('/api/visits')` con **ruta relativa**, no una URL absoluta. Como Nginx sirve el frontend y hace de proxy del backend en el mismo origen (`http://localhost:8080`), no hace falta configurar CORS.
- Incluye un botón que dispara 6 peticiones seguidas a `/api/health` y muestra qué backend respondió cada una — es la forma más rápida de demostrar el balanceo de carga en vivo frente a la audiencia.

4.3 Nginx (`nginx/nginx.conf.tpl`)

Este archivo **no es un `nginx.conf` normal**: tiene sintaxis de plantilla de Terraform (`${ }` para interpolar, `%{ for }` para iterar). Terraform lo procesa con la función `templatefile()` y sube el resultado ya renderizado al contenedor. Esto es exactamente lo que verás en `terraform/nginx.tf`.

4.4 MySQL (`mysql/init.sql`)

Un script SQL estándar que la imagen oficial de MySQL ejecuta automáticamente la primera vez que inicializa su volumen de datos, montado en `/docker-entrypoint-initdb.d/`.

5. Parte B — El código Terraform, explicado

Todo vive en `terraform/`. Revisa cada archivo mientras avanzas — todos están comentados dentro del proyecto; aquí se explica el porqué de cada decisión.

5.1 `versions.tf`

```
versions.tf

terraform {
  required_version = ">= 1.5.0"
  required_providers {
    docker = {
      source  = "kreuzwerker/docker"
      version = "~> 4.0"
    }
  }
}

provider "docker" {}
```

`kreuzwerker/docker` es el proveedor comunitario más usado para gestionar Docker con Terraform. Al no pasarle ningún argumento (`provider "docker" {}`), usa el contexto activo de Docker en tu máquina — el mismo que usarías con `docker ps`.

5.2 `variables.tf`

Define réplicas del backend, puerto de Nginx, y credenciales de MySQL. Las credenciales llevan `sensitive = true`, para que Terraform las oculte en la salida de `plan/apply` — aunque, como se explica en la sección de producción, tener un `default` en una variable sensible solo es aceptable en un laboratorio local como este.

5.3 `locals.tf`

Calcula una sola vez la lista de alias del backend (`backend-1`, `backend-2`, ...) a partir de `var.backend_replica_count`, para que `backend.tf` y `nginx.tf` nunca queden desincronizados entre sí.

5.4 network.tf

Una red Docker privada (`iac-workshop-net`) es el "cableado" que conecta todo. Gracias al DNS embebido de Docker, cualquier contenedor puede resolver a otro por su alias.

5.5 mysql.tf

Tres recursos: la imagen, un **volumen nombrado** (persistencia) y el contenedor. Aquí aparecen dos mecanismos importantes:

- `healthcheck { ... } + wait = true`: Terraform no marca este recurso como aplicado hasta que MySQL responda "healthy" — no basta con que el proceso haya arrancado.
- `depends_on` (en `backend.tf`) obliga a que el backend espere a que esto ocurra.

5.6 backend.tf

El bloque más importante para entender `count`:

```
backend.tf (fragmento)
resource "docker_container" "backend" {
  count = var.backend_replica_count
  name  = "workshop-backend-${count.index + 1}"
  ...
}
```

Con `backend_replica_count = 2`, Terraform crea **dos recursos independientes** en el grafo y en el estado: `docker_container.backend[0]` y `docker_container.backend[1]`. Esto será clave en la Parte D.

También fíjate en el bloque `triggers` del `docker_image`: calcula un hash SHA1 de todos los archivos de `backend/` para que Terraform sepa que debe reconstruir la imagen si el código cambió, aunque el nombre de la imagen siga siendo el mismo.

5.7 frontend.tf

Mismo patrón que el backend, pero sin `count` (una sola réplica del frontend en este taller).

5.8 nginx.tf

La pieza más "IaC" de todo el proyecto:

```
nginx.tf (fragmento)
upload {
  content = templatefile("${path.module}/../nginx/nginx.conf.tpl", {
    backend_hosts = local.backend_aliases
  })
  file = "/etc/nginx/nginx.conf"
}
```


Terraform genera el `nginx.conf` en el momento del `apply`, a partir de la plantilla y de la lista actual de réplicas del backend, y lo sube al contenedor antes de arrancarlo. Si cambias el número de réplicas, este contenido cambia, y — como el archivo se sube al crear el contenedor, no se puede parchar en caliente — Terraform **destruye y recrea** el contenedor de Nginx. Es el mismo principio de infraestructura inmutable que se explicó en la sesión conceptual de IaC, ahora visible en la práctica.

5.9 outputs.tf

Expone la URL de la app, los nombres de los contenedores del backend y el nombre del volumen de MySQL — información que vas a usar en los pasos siguientes sin tener que recordarla de memoria.

6. Parte C — Ejecutar el taller paso a paso

Todos los comandos se ejecutan dentro de `terraform/`.

```
cd workshop/terraform
```

Paso 1 — Inicializar

```
terraform init
```

Esto descarga el plugin del proveedor `kreuzwerker/docker` y prepara el directorio `.terraform/`. Solo hace falta repetirlo si cambias de proveedor o de versión.

Paso 2 — Formatear y validar

```
terraform fmt
terraform validate
```

`fmt` normaliza el estilo del código; `validate` revisa sintaxis y referencias internas sin necesitar Docker activo.

Paso 3 — Planificar

```
terraform plan
```

Vas a ver algo como `Plan: 9 to add, 0 to change, 0 to destroy.` (1 red, 1 imagen+contenedor de MySQL, 1 volumen, 2 imágenes+contenedores de backend, 1 imagen+contenedor de frontend, 1

imagen+contenedor de Nginx). **Nada se crea todavía.** Este es el momento de leer con calma qué va a pasar — el hábito más importante que se practicó en la clase conceptual.

Paso 4 — Aplicar

```
terraform apply
```

Confirma con `yes`. Vas a notar que MySQL tarda unos segundos extra en completarse: es el `wait_timeout` del healthcheck haciendo su trabajo antes de que arranque el backend.

Al terminar, Terraform imprime los `outputs`:

```
app_url = "http://localhost:8080"
api_health_url = "http://localhost:8080/api/health"
backend_container_names = [
  "workshop-backend-1",
  "workshop-backend-2",
]
...
```

Paso 5 — Verificar

```
docker ps          # deberías ver 5 contenedores corriendo
curl http://localhost:8080/api/health
curl http://localhost:8080/api/visits
```

Abre `http://localhost:8080` en el navegador: deberías ver la interfaz de React, el contador de visitas leyendo de MySQL, y el botón para probar el balanceo de carga entre `backend-1` y `backend-2`.

7. Parte D — El corazón del taller: practicar el archivo de estado

Esta es la sección que **no tiene equivalente real en Docker Compose**, y es la razón de ser de este taller. Compose puede levantar los mismos cuatro servicios con un `docker-compose.yml`, pero no mantiene un registro persistente y consultable de "qué es cada recurso, con qué atributos, y cómo se relaciona con los demás" — solo compara, de forma superficial, si la configuración de cada servicio cambió respecto a la última vez, usando un hash interno en las etiquetas del contenedor. No hay comando equivalente a `plan`, no hay forma de renombrar una entrada sin destruir el recurso real, y no hay forma de "adoptar" un contenedor creado por fuera.

Ejecuta cada comando y observa el resultado antes de seguir al siguiente.

7.1 Inspeccionar el estado

```
terraform state list
```

Verás la lista completa de direcciones de recursos que Terraform administra, incluyendo el índice de cada réplica:

```
docker_container.backend[0]
docker_container.backend[1]
docker_container.frontend
docker_container.mysql
docker_container.nginx
docker_image.backend
docker_image.frontend
docker_image.mysql
docker_image.nginx
docker_network.workshop_net
docker_volume.mysql_data
```

```
terraform state show "docker_container.backend[0]"
```

Muestra **todos** los atributos que Terraform conoce de ese contenedor específico: su ID real de Docker, IP interna, variables de entorno, red, etc. Esto es información que Docker Compose no expone de forma consultable ni persistente entre ejecuciones.

```
terraform show
```

Vuelca el estado completo, legible por humanos — útil para una auditoría rápida de "qué existe ahora mismo, según Terraform".

7.2 Idempotencia: re-aplicar sin cambios

```
terraform plan
```

Debería mostrar `No changes. Your infrastructure matches the configuration.` Esto es la idempotencia declarativa en acción: como nada cambió en el código ni en la infraestructura real, Terraform no propone ninguna acción.

7.3 Provocar y detectar drift

Drift es cuando la infraestructura real se desvía de lo que el código declara, típicamente por un cambio manual. Vamos a simularlo borrando un contenedor directamente con el CLI de Docker, **sin pasar por Terraform** — como haría alguien aplicando un "arreglo rápido" en caliente:

```
docker rm -f workshop-backend-2
```

Ahora pregúntale a Terraform qué opina:

```
terraform plan
```

Vas a ver que Terraform detecta que `docker_container.backend[1]` ya no existe en la realidad, aunque su estado diga que debería existir, y propone recrearlo:

```
# docker_container.backend[1] will be created
+ resource "docker_container" "backend" { ... }

Plan: 1 to add, 0 to change, 0 to destroy.
```

Esto es exactamente lo que Docker Compose no puede hacer. Si borras un contenedor gestionado por `docker-compose.yml` y ejecutas `docker compose up` de nuevo, Compose sí lo vuelve a crear porque compara "qué servicios del archivo YAML no tienen un contenedor corriendo" — pero no tiene un registro histórico y consultable que te permita, por ejemplo, saber qué IP tenía antes, auditar cuándo se creó, o detectar drift en atributos que no sean "¿existe o no existe?" (por ejemplo, alguien cambió manualmente una variable de entorno con `docker update`).

7.4 Corregir el drift

```
terraform apply
```

Terraform recrea únicamente el contenedor faltante — no toca los demás recursos, porque el resto del estado sigue coincidiendo con la realidad.

7.5 Reemplazar un recurso puntual con `-replace`

A veces no hay drift, pero quieres forzar la recreación de un recurso específico (por ejemplo, "sospecho que este contenedor tiene un problema, quiero una instancia nueva y limpia"):

```
terraform apply -replace="docker_container.backend[0]"
```

Terraform destruye y vuelve a crear **solo** `backend-1`, dejando `backend-2` intacto. (Esta bandera reemplaza al antiguo comando `terraform taint`, que sigue existiendo pero está desaconsejado desde Terraform 0.15.2.)

7.6 Renombrar una dirección sin destruir infraestructura real

Imagina que decides renombrar el recurso `docker_container.frontend` a `docker_container.web` en el código, porque el nombre "frontend" ya no te conviene. Si solo editas el `.tf` y aplicas, Terraform planeará **destruir** el contenedor viejo y **crear** uno nuevo — aunque en la práctica sea exactamente el mismo contenedor. `state mv` evita esa recreación innecesaria:

```
terraform state mv docker_container.frontend docker_container.web
```

Ahora sí puedes renombrar el bloque `resource "docker_container" "frontend"` a `"web"` en `frontend.tf`, y `terraform plan` mostrará "sin cambios" en vez de un destroy/create. Vuelve a dejarlo como `"frontend"` (y repite el `state mv` en sentido inverso) para no romper el resto del taller.

7.7 Importar un recurso creado manualmente

Este es, quizás, el ejercicio más ilustrativo de todos. Crea un contenedor **totalmente fuera de Terraform**, como si fuera un recurso heredado que alguien levantó a mano hace tiempo:

```
docker network connect iac-workshop-net --alias redis-manual $(docker run -d --name manual-redis redis:7-alpine)
```

(O más simple: `docker run -d --name manual-redis --network iac-workshop-net redis:7-alpine`)

Ahora escribe el bloque que **debería** describir ese contenedor en un archivo nuevo, por ejemplo `terraform/redis.tf`:

```
terraform/redis.tf
resource "docker_image" "redis" {
  name = "redis:7-alpine"
}

resource "docker_container" "redis" {
  name = "manual-redis"
  image = docker_image.redis.image_id

  networks_advanced {
    name = docker_network.workshop_net.name
  }
}
```

Y adóptalo en el estado, sin recrearlo:

```
terraform import docker_container.redis $(docker inspect -f '{{.Id}}' manual-redis)
```

Importante: no existe un `terraform import docker_image...` equivalente para la imagen. El recurso `docker_image` del proveedor `kreuzwerker/docker` **no soporta import**. En la práctica no hace falta: como la

imagen `redis:7-alpine` ya existe localmente (Docker la descargó al ejecutar `docker run`), Terraform simplemente la "creará" en el estado sin volver a descargarla — es un efecto inofensivo, no una recreación real.

```
terraform plan
```

El plan mostrará `docker_image.redis` como recurso a crear (normal, ver nota anterior) y, si todo salió bien, `docker_container.redis` con pocos o ningún cambio (puede pedirte ajustar algún atributo menor para que el código coincida exactamente con lo importado — es normal, forma parte del ejercicio). Aplica con `terraform apply` para completar la adopción.

Docker Compose no tiene ningún comando equivalente a `terraform import`: solo puede gestionar lo que él mismo creó. Y este mismo ejercicio deja una lección extra: no todos los tipos de recurso de un proveedor soportan import — siempre conviene revisar la documentación del proveedor antes de asumirlo.

Cuando termines el ejercicio, limpia:

```
rm terraform/redis.tf
docker rm -f manual-redis
terraform state rm docker_container.redis docker_image.redis
```

`terraform state rm` saca un recurso del estado **sin destruirlo en la realidad** — útil cuando quieres que Terraform "deje de administrar" algo sin borrarlo (por ejemplo, si vas a traspasarlo a otro equipo o a otro workspace).

7.8 Resumen de la comparación

Capacidad	Docker Compose	Terraform (proveedor docker)
Detecta si el YAML/HCL cambió	Sí (hash de configuración)	Sí
Detecta drift causado por cambios manuales fuera de la herramienta	No	Sí (<code>terraform plan</code> re-consulta la realidad)
Vista previa de cambios antes de aplicar (dry-run con diff explícito)	No de forma nativa	Sí (<code>terraform plan</code>)
Consultar atributos completos de un recurso ya aplicado	Limitado (<code>docker inspect</code> manual)	Sí (<code>terraform state show</code>)
Renombrar una definición sin recrear el recurso real	No	Sí (<code>terraform state mv</code>)
Adoptar (importar) un recurso creado por fuera	No	Sí (<code>terraform import</code>)

Capacidad	Docker Compose	Terraform (proveedor docker)
Reemplazar un recurso puntual bajo demanda	No	Sí (<code>apply -replace</code>)
Bloqueo para evitar aplicaciones concurrentes	No	Sí, con backend remoto (ver sección de producción)

8. Parte E — Escalar el backend

```
# En terraform/terraform.tfvars (créalo a partir de terraform.tfvars.example)
backend_replica_count = 4
```

```
terraform plan
```

El plan mostrará: 2 contenedores nuevos de backend (`workshop-backend-3`, `workshop-backend-4`) y que `docker_container.nginx` se va a destruir y recrear — porque su configuración (`nginx.conf`, generado por `templatefile()`) cambió al incluir las nuevas réplicas.

```
terraform apply
```

Vuelve a probar el botón de balanceo de carga en el frontend: ahora deberías ver hasta 4 hostnames distintos repartiéndose las peticiones.

9. Parte F — Destruir el entorno

```
terraform destroy
```

Confirma con `yes`. Esto elimina los 5 contenedores, las 3 imágenes construidas/descargadas, la red y también el volumen de datos de MySQL (a menos que hayas activado `prevent_destroy`, ver `mysql.tf`).

10. Consejos si esto se llevara a producción

El proveedor `docker` de Terraform es excelente para aprender, para entornos de desarrollo local y para demos — **no está pensado para producción**. Estos son los ajustes que un equipo debería hacer antes de dar ese salto:

1. **Cambiar de orquestador.** El proveedor `docker` administra contenedores en un solo host Docker: no reinicia nada si el host se cae, no distribuye carga entre varias máquinas y no tiene autoescalado. En producción, el mismo enfoque declarativo se aplica con Terraform sobre Kubernetes (`kubernetes/helm` providers), Amazon ECS, o el orquestador que la organización use — Terraform sigue siendo la capa de definición, pero el motor de ejecución cambia.
2. **Backend remoto de estado, con bloqueo.** Este taller usa un `terraform.tfstate` local en tu disco — perfecto para practicar solo, pero inviable en equipo: dos personas aplicando al mismo tiempo pueden corromperlo. En producción: backend remoto (S3+DynamoDB, Terraform Cloud, GCS) con cifrado y locking, como se vio en la clase de gestión del estado.
3. **Nunca variables sensibles con `default`.** En `variables.tf` de este taller, `mysql_root_password` y `mysql_password` tienen valores por defecto para que el taller funcione con un solo `terraform apply`. En un proyecto real, esas variables no deberían tener `default`, no deberían vivir en un `.tfvars` versionado en Git, y deberían inyectarse desde un gestor de secretos (Vault, AWS Secrets Manager, GCP Secret Manager) en tiempo de ejecución del pipeline.
4. **Tags de imagen inmutables, nunca `:latest`.** Este taller usa `mysql:8.0` y `nginx:1.27-alpine` (ya son versiones fijas, buena práctica), pero las imágenes propias (`iac-workshop-backend:local`) deberían versionarse explícitamente (`myapp-backend:1.4.2` o el hash del commit) para poder hacer rollback con certeza.
5. **Límites de recursos por contenedor.** Añade `memory` y `cpu_shares` a cada `docker_container` (o el equivalente en el orquestador de producción) para que un solo servicio no pueda agotar los recursos del host.
6. **TLS/HTTPS en el punto de entrada.** Aquí Nginx expone HTTP plano en el puerto 8080 por simplicidad. En producción, el balanceador debe terminar TLS con certificados gestionados (y redirigir 80 → 443).
7. **Backups del dato, o mejor aún, base de datos gestionada.** Un contenedor de MySQL con un volumen local no tiene backups automáticos ni alta disponibilidad. En producción conviene un servicio de base de datos gestionado (RDS, Cloud SQL, Azure Database) o, si se mantiene en contenedores, un proceso explícito de respaldo del volumen.
8. **`lifecycle { prevent_destroy = true }` en recursos con estado crítico.** Ver el comentario en `mysql.tf`: es una red de seguridad real contra un `terraform destroy` accidental sobre datos productivos.
9. **Pipeline de CI/CD con aprobación humana,** políticas como código (Sentinel/OPA) y escaneo de imágenes (Trivy, Checkov sobre los Dockerfiles) antes de construir y desplegar — retomando lo visto en la clase de gobernanza para producción.
10. **Usuario no root en los contenedores.** El `Dockerfile` del backend ya usa `USER node`; audita que cualquier imagen nueva del proyecto haga lo mismo.

11. **Monitoreo y logging centralizado.** Los contenedores locales de este taller no envían logs ni métricas a ningún lado; en producción, intégralos con tu stack de observabilidad (Prometheus/Grafana, CloudWatch, Datadog, etc.).
12. **Separar ambientes con estados completamente distintos** (no solo variables distintas) — dev, staging y producción no deberían ni siquiera compartir el mismo backend de estado, para limitar el "radio de explosión" de un error.

11. Solución de problemas comunes

Síntoma	Causa probable	Solución
Error: Cannot connect to the Docker daemon	Docker Desktop/Engine no está corriendo	Inicia Docker y confirma con <code>docker ps</code>
Error: Bind for 0.0.0.0:8080 failed: port is already allocated	Otro proceso ya usa el puerto 8080	Cambia <code>nginx_host_port</code> en <code>terraform.tfvars</code>
<code>terraform init</code> no descarga el proveedor	Sin acceso a <code>registry.terraform.io</code>	Revisa la conexión a internet o la configuración de proxy/firewall corporativo
El <code>apply</code> se queda esperando en MySQL	El healthcheck no pasa a "healthy" a tiempo	Revisa <code>docker logs workshop-mysql</code> ; si tu equipo es lento, sube <code>wait_timeout</code> en <code>mysql.tf</code>
El backend no puede conectar a MySQL	Variables de entorno mal propagadas, o MySQL aún inicializando	<code>docker logs workshop-backend-1</code> ; confirma que <code>DB_HOST=mysql</code> resuelve con <code>docker exec workshop-backend-1 ping mysql</code>
<code>volumes.0.host_path</code> must be an absolute path (Windows)	Rutas relativas mal resueltas en Windows	Usa <code>abspath()</code> como en este proyecto, y prueba con barras / en vez de \
Cambié el código del backend pero la imagen no se reconstruye	El hash de <code>triggers</code> no cambió, o hay caché de build	Confirma que guardaste el archivo; como último recurso, <code>docker rmi iac-workshop-backend:local</code> y vuelve a aplicar

12. Retos adicionales (para quien quiera ir más allá)

1. Agrega un contenedor de **Redis** como caché y conéctalo a la misma red — practica todo el ciclo `image` → `container` → `networks_advanced` de nuevo, ahora sin ayuda.
2. Añade un `output` que muestre la IP interna de cada contenedor con `docker_container.backend[*].network_data`.

3. Convierte `backend.tf` en un **módulo Terraform reutilizable** (`modules/app-replica/`), parametrizado por nombre de servicio, imagen y variables de entorno.
4. Activa `lifecycle { prevent_destroy = true }` en `docker_volume.mysql_data` y comprueba que `terraform destroy` falla como se espera; luego quítalo para poder limpiar el entorno.
5. Instala `tflint` y corre `terraform fmt -check + tflint` como si fueran un paso de CI.
6. (Avanzado) Escribe una política simple con Open Policy Agent/Conftest que rechace cualquier `docker_container` sin el argumento `restart` definido.

13. Referencia rápida de comandos

Comando	Qué hace
<code>terraform init</code>	Descarga providers y prepara el directorio de trabajo
<code>terraform fmt</code>	Normaliza el formato del código
<code>terraform validate</code>	Revisa sintaxis y referencias, sin tocar Docker
<code>terraform plan</code>	Calcula y muestra el plan de ejecución, sin aplicar nada
<code>terraform apply</code>	Ejecuta el plan (pide confirmación)
<code>terraform destroy</code>	Elimina todos los recursos gestionados
<code>terraform state list</code>	Lista las direcciones de todos los recursos en el estado
<code>terraform state show <dirección></code>	Muestra todos los atributos conocidos de un recurso
<code>terraform show</code>	Vuelca el estado completo en formato legible
<code>terraform state mv <origen> <destino></code>	Renombra una dirección sin destruir el recurso real
<code>terraform state rm <dirección></code>	Deja de administrar un recurso, sin destruirlo
<code>terraform import <dirección> <id></code>	Adopta un recurso ya existente en el estado
<code>terraform apply -replace="<dirección>"</code>	Fuerza la recreación de un recurso puntual
<code>terraform output</code>	Muestra los valores de salida definidos en <code>outputs.tf</code>

Fin del taller. El proyecto completo (aplicaciones + Terraform) está incluido en el archivo comprimido que acompaña esta guía — solo necesitas descomprimirlo.